

TEK4090

Introduction to Modern Control

Lecture 8: Model Predictive Control

Mathias Hudoba de Badyn

October 22, 2025



This presentation is adapted from Computational Control by Saverio Bolognani, and licensed under a Creative Commons Attribution-ShareAlike 4.0 International License

Recap

1. System identification

Reading Assignment

1. **Rawlings, Mayne, and Diehl:** Ch 1,2,7
2. **Coulson *et al.*:** In the Shallows of the DeePC by Coulson

Recap: LQR

Constrained linear-quadratic optimal control

$$\begin{aligned} \min_{\substack{u_0, \dots, u_{K-1} \\ x_0, \dots, x_K}} \quad & \sum_{k=0}^{K-1} \underbrace{(x_k^T Q x_k + u_k^T R u_k)} + x_K^T S x_K \\ \text{subject to} \quad & \begin{cases} x_{k+1} = Ax_k + Bu_k & \text{for } k = 0, \dots, K-1 \\ x_0 = X_0 \end{cases} \end{aligned}$$

$$\left(\begin{array}{l} b \leq x_k \leq c \\ 0 \leq u_k \leq 1 \\ \text{not covered} \end{array} \right)$$

- Solved by backwards induction: **Dynamic Programming**
- ▶ quadratic cost $\rightarrow$ quadratic value function (cost-to-go)
 - ▶ system constraints
 - ▶ recursive dynamic programming: Ricatti Equation $\leftarrow$

Beyond LQR

Constrained linear-quadratic optimal control

$$\begin{aligned} \min_{\substack{u_0, \dots, u_{K-1} \\ x_0, \dots, x_K}} \quad & \sum_{k=0}^{K-1} \left(\underline{x_k^T Q x_k} + u_k^T R u_k \right) + x_K^T S x_K \\ \text{subject to} \quad & x_{k+1} = A x_k + B u_k \quad \text{for } k = 0, \dots, K-1 \\ & x_0 = X_0 \\ & x_k \in \mathcal{X}_k \quad (\text{polytope}) \\ & u_k \in \mathcal{U}_k \quad (\text{polytope}) \end{aligned}$$

drive the state to zero
 $\mapsto x = z - z_0$

No backward induction, one single program with

- ▶ quadratic cost
- ▶ convex constraints
- ▶ linear equality constraints

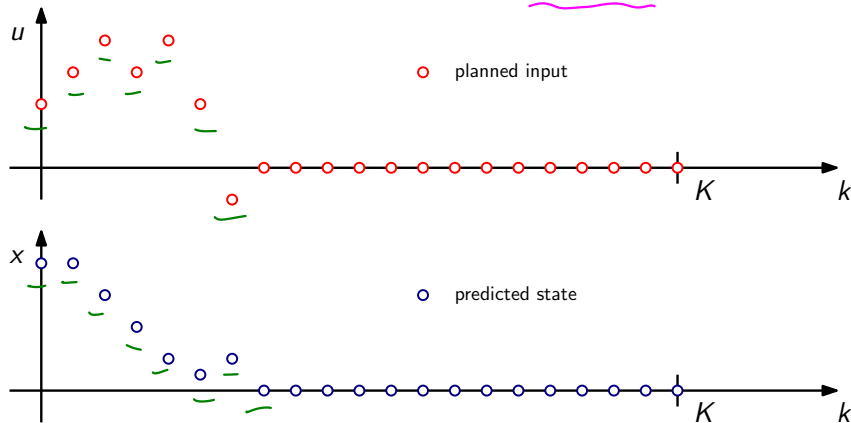
Open-loop optimal control

By solving the optimization **in one shot**, we get

$$\underline{u_0^*}, \underline{u_1^*}, \dots, \underline{u_{K-1}^*} \in \mathbb{R}^m$$

instead of a state feedback

$$u_0(x), u_1(x), \dots, u_{K-1}(x) \quad \text{like} \quad \underline{u_t = \Gamma_t x_t} \text{ (LQR)}$$



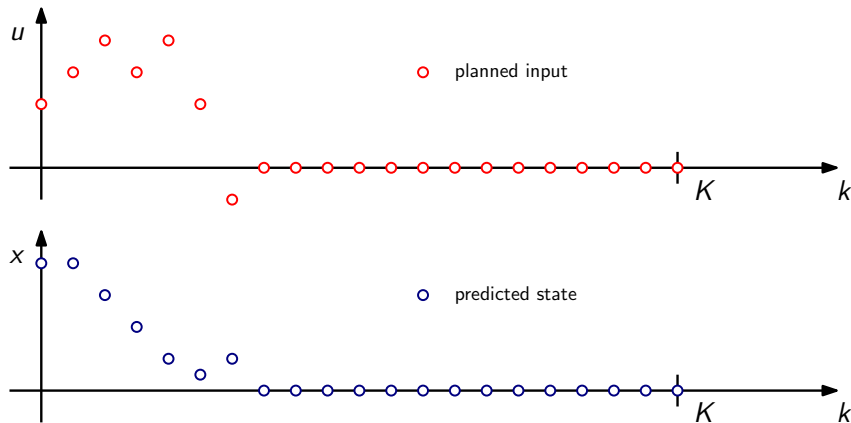
Open-loop optimal control

By solving the optimization **in one shot**, we get

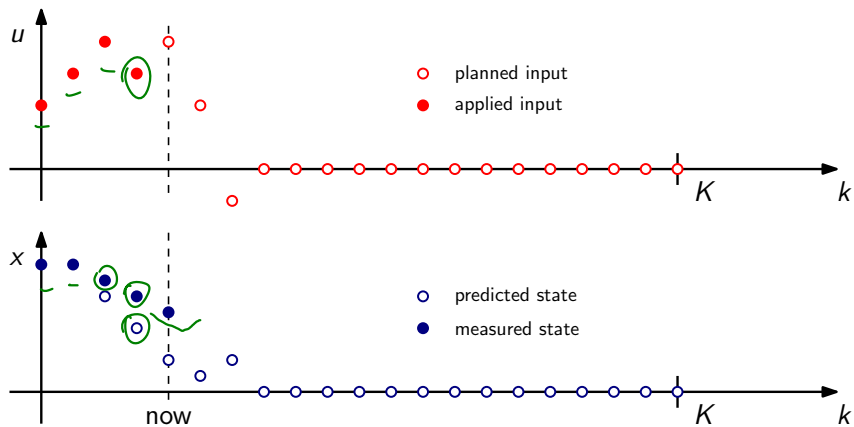
$$u_0^*, u_1^*, \dots, u_{K-1}^* \in \mathbb{R}^m$$

instead of a state feedback

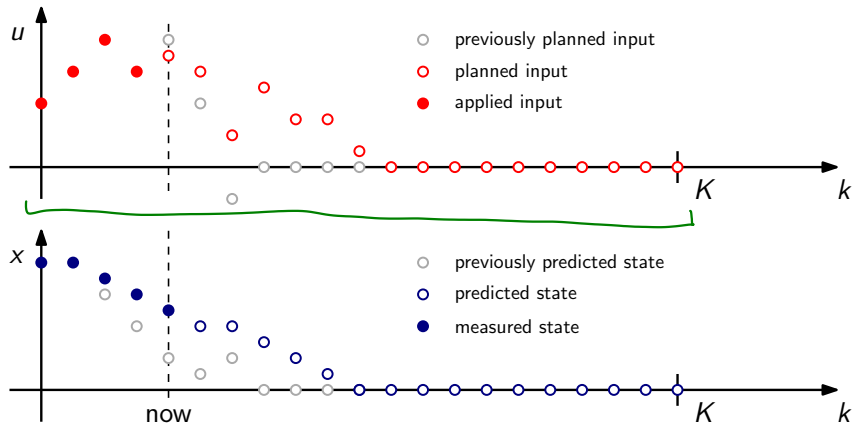
$$u_0(x), u_1(x), \dots, u_{K-1}(x) \quad \text{like} \quad u_t = \Gamma_t x_t \text{ (LQR)}$$



→ **open-loop** optimal control strategy

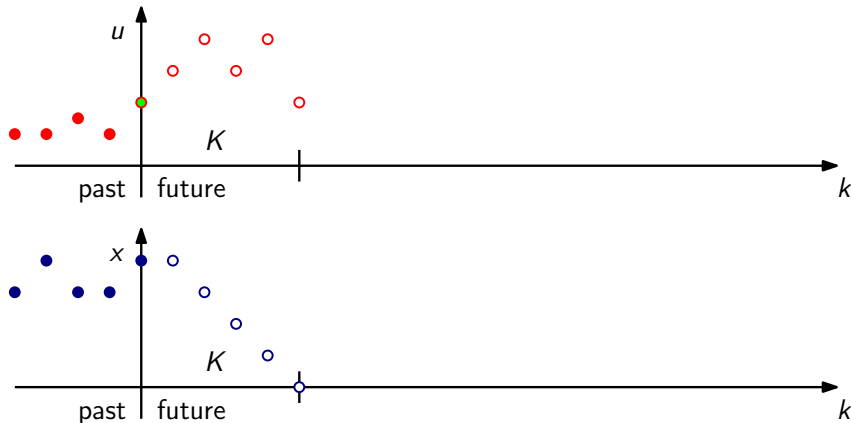


What if a **disturbance/model mismatch** takes you elsewhere?



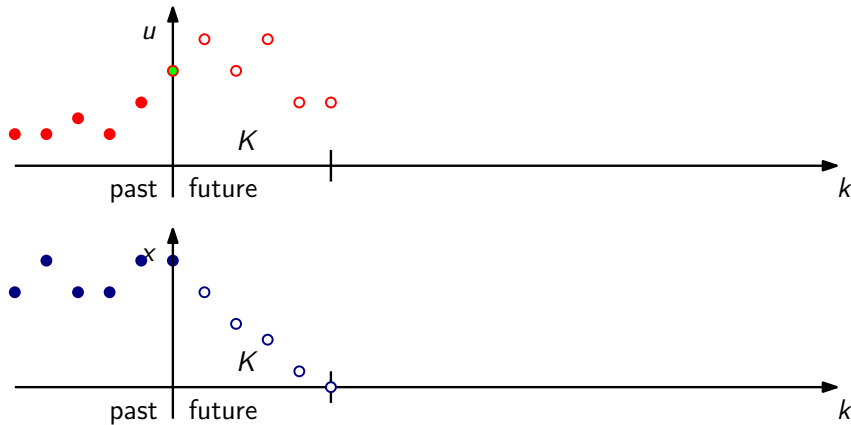
Recompute!

- ▶ Incorporate new information about the current state
- ▶ Markovian system: current state is all you need to know to compute the optimal control for future
- ▶ Previously computed optimal trajectory is invalid
- ▶ Computation of the new optimal trajectory needs to happen in 1 time step



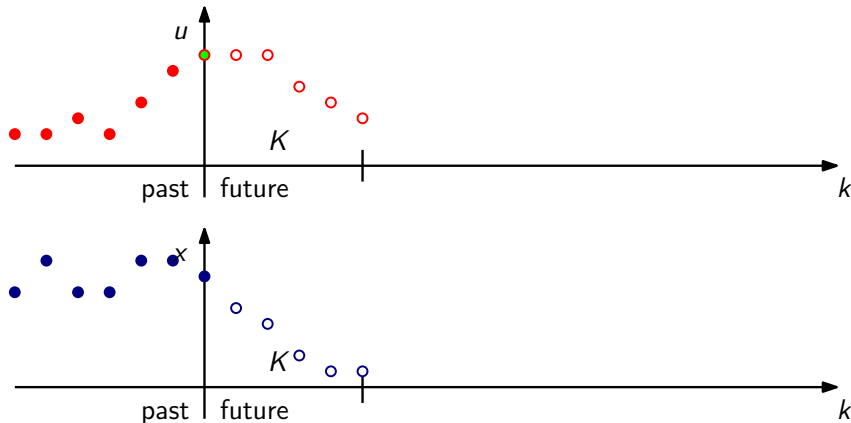
Receding horizon

- ▶ Apply only one control input, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



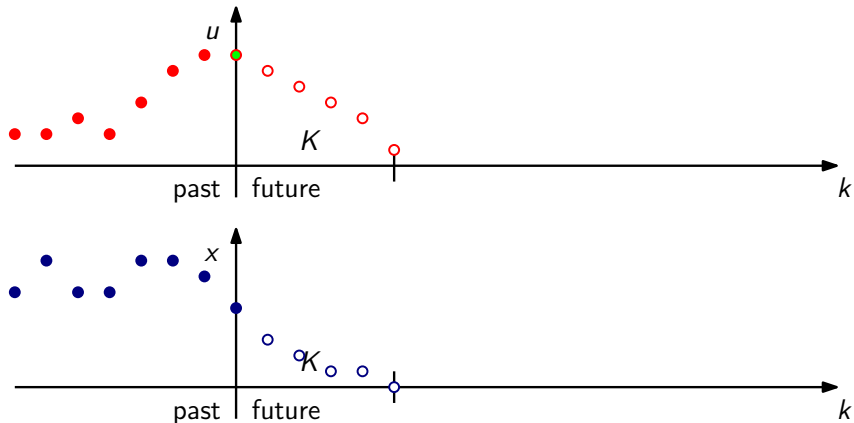
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



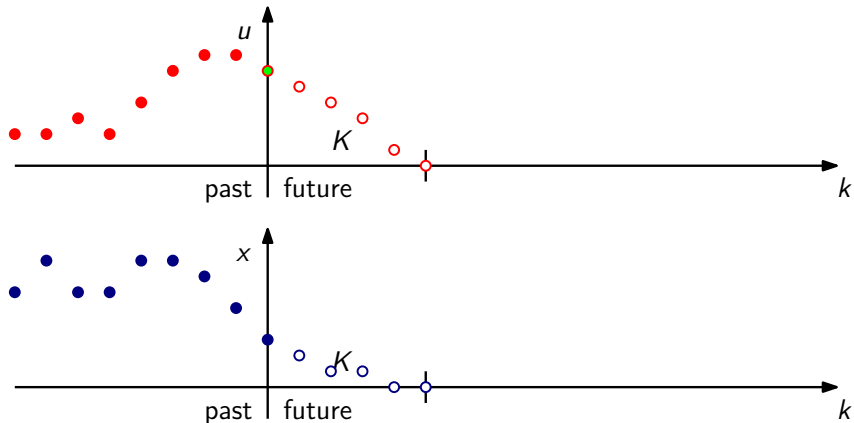
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



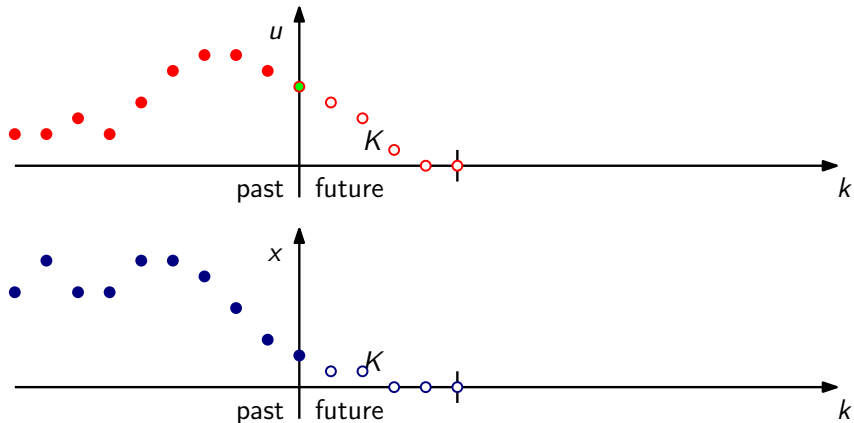
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



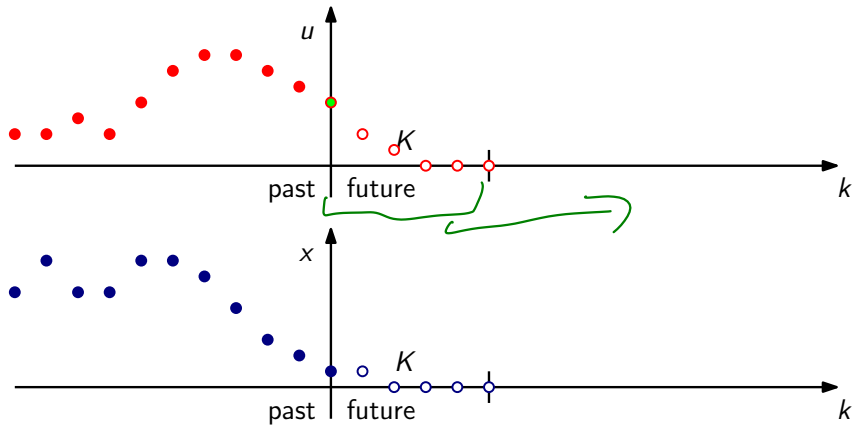
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



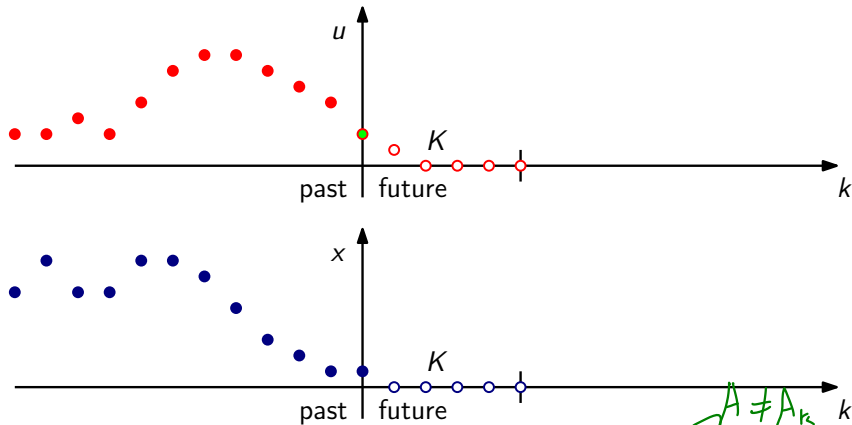
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0

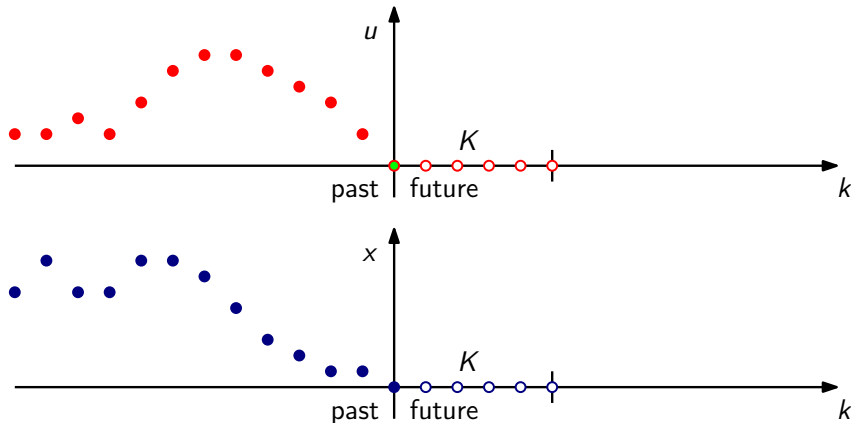


Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0

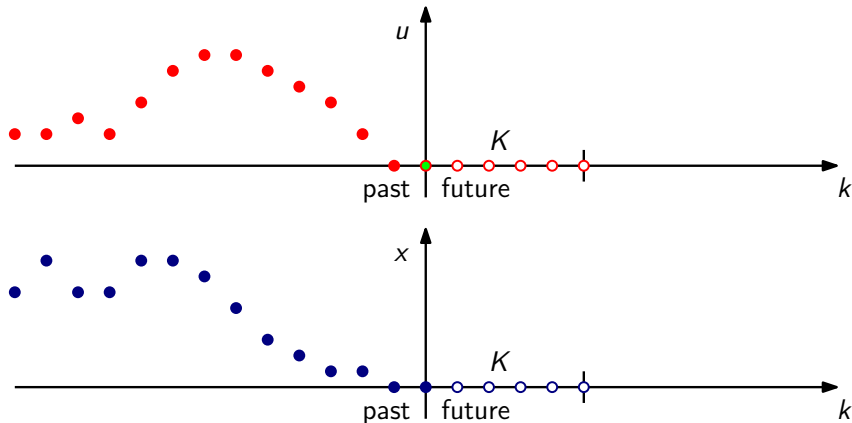
$$x_{n+1} = A x_n + B u_n$$

$A \neq A_n$



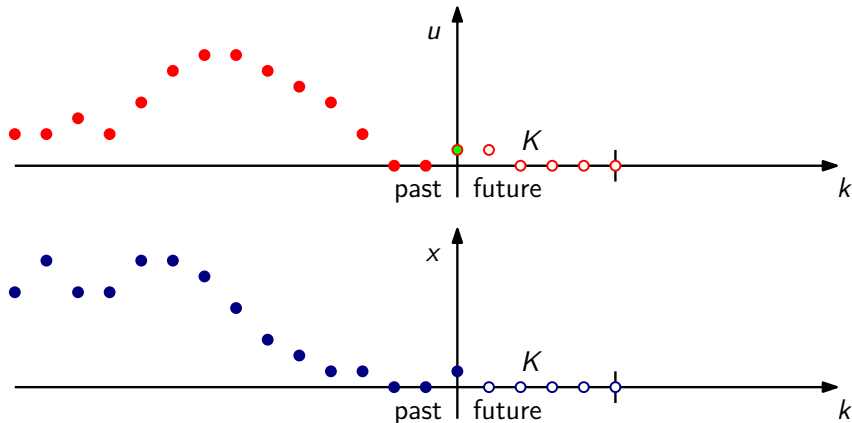
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



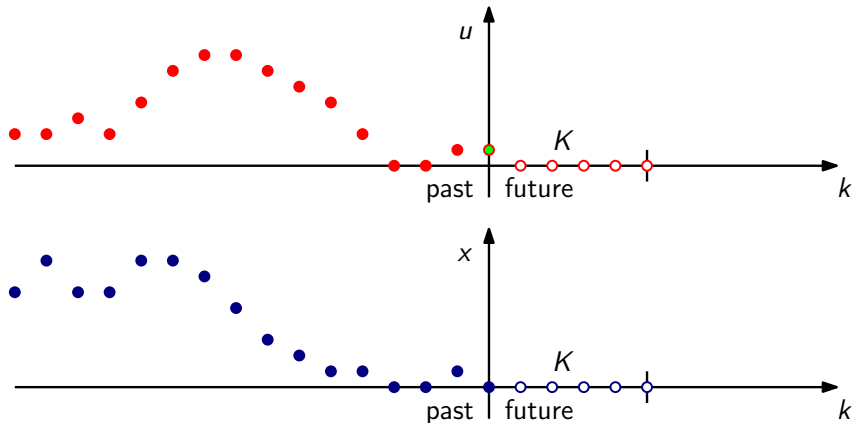
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



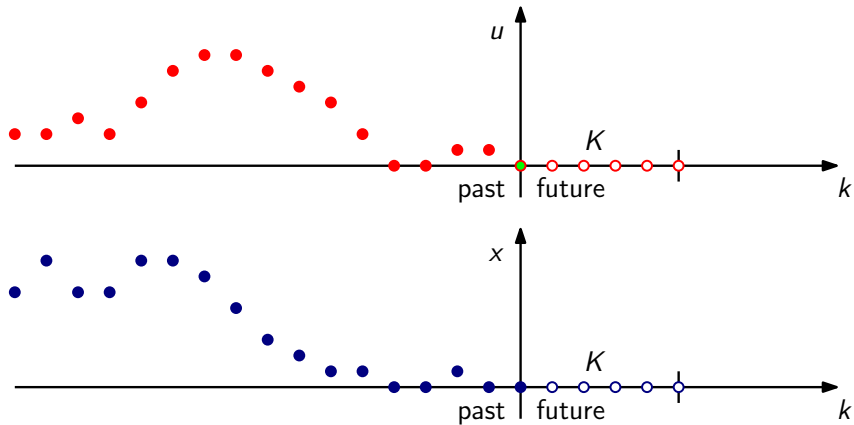
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length → **receding horizon**
- ▶ If the system is time invariant → same **optimization problem**, different x_0



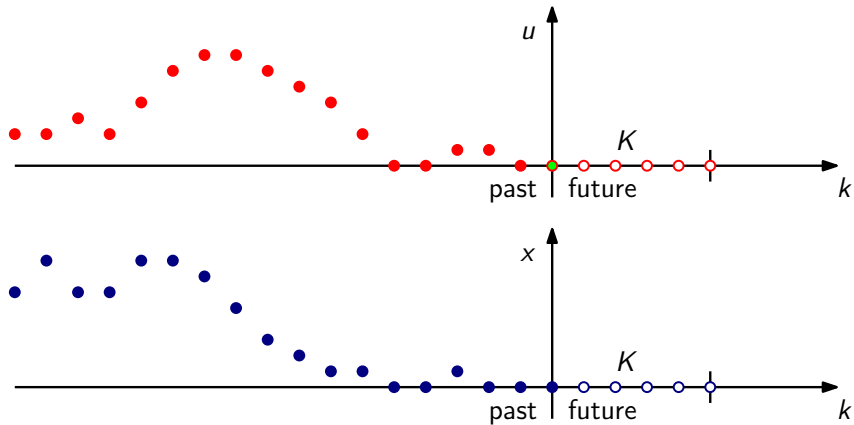
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



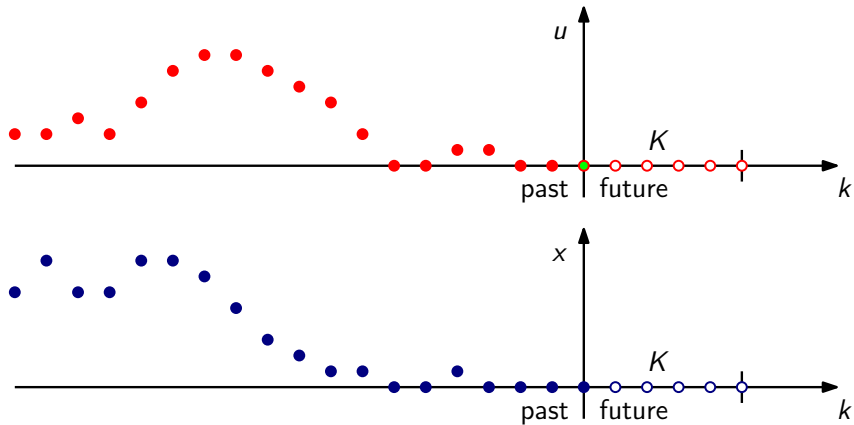
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



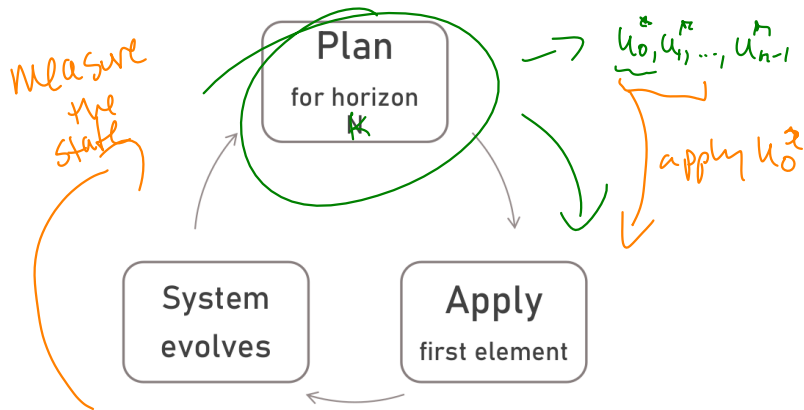
Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same **optimization problem**, different x_0



Receding horizon

- ▶ Apply only **one control input**, u_0^*
- ▶ Use new state measurement as new **initial conditions**
- ▶ Same horizon length $\rightarrow$ **receding horizon**
- ▶ If the system is time invariant $\rightarrow$ same optimization problem, different x_0

Receding-horizon control

The resulting **open-loop optimal control problem** corresponds to a *parametric optimization problem* parametrized in the **initial state x** .

Parametric optimization

$$\underbrace{u_0^*(x)} \text{ determined by } \min_{u, x} \sum_{k=0}^{K-1} \underbrace{g_k(x_k, u_k)} + \underbrace{g_K(x_K)}$$

subject to $x_{k+1} = f(x_k, u_k)$

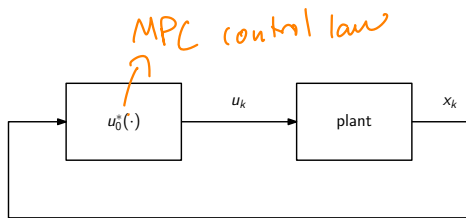
$$\underbrace{x_0 = x}$$

$$x_k \in \mathcal{X}_k$$

$$u_k \in \mathcal{U}_k$$

Monotone Operator
Theory

Feedforward or feedback?



What do we know about the optimization problem?

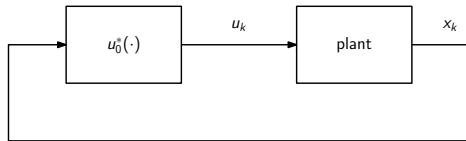
- ▶ Under **continuity** assumptions of g and f , the minimum cost $V_K(\mathbf{x}, \mathbf{u})$ is a continuous function of x_0 .
- ▶ Under **compactness** assumptions on the constraints, the parametric optimization problem has a solution when it's feasible.
- ▶ However, the map $x \mapsto u_0^*(x)$ is not necessarily continuous

MPC control law

MPC is a **memory-less, nonlinear, time-invariant feedback control law**.

$\hookrightarrow u_0^(x)$ only depends on x*

Feedforward or feedback?



What do we know about the optimization problem?

- ▶ Under **continuity** assumptions of g and f , the minimum cost $V_K(\mathbf{x}, \mathbf{u})$ is a continuous function of x_0 .
- ▶ Under **compactness** assumptions on the constraints, the parametric optimization problem has a solution when it's feasible.
- ▶ However, the map $x \mapsto u_0^*(x)$ is not necessarily continuous

MPC control law

MPC is a **memory-less, nonlinear, time-invariant feedback control law**.

Question: Can I achieve zero tracking error with an MPC controller?

$$\min_{\mathbf{u}, \mathbf{x}} \sum_{k=0}^{K-1} \left(\mathbf{x}_k^\top \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k \right) + \mathbf{x}_K^\top \mathbf{S} \mathbf{x}_K, \quad \mathbf{Q}, \mathbf{S} \succeq 0, \mathbf{R} \succ 0$$

row $\mathbf{x}$

subject to $\mathbf{x}_{k+1} = \mathbf{A} \mathbf{x}_k + \mathbf{B} \mathbf{u}_k$

Finite-time LQR

- ▶ Backward computation of $\mathbf{P}_K, \mathbf{P}_{K-1}, \dots, \mathbf{P}_1$
- ▶ Linear feedback $\mathbf{u}_k = \mathbf{\Gamma}_k \mathbf{x}_k$ with $\mathbf{\Gamma}_k = -(\mathbf{R} + \mathbf{B}^\top \mathbf{P}_{k+1} \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{P}_{k+1} \mathbf{A}$

$$\begin{aligned} \mathbf{u}_0^{\text{LQR}} &= -(\mathbf{R} + \mathbf{B}^\top \mathbf{P}_1 \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{P}_1 \mathbf{A} \mathbf{x}_0 \\ \mathbf{u}_1^{\text{LQR}} &= -(\mathbf{R} + \mathbf{B}^\top \mathbf{P}_2 \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{P}_2 \mathbf{A} (\mathbf{A} \mathbf{x}_0 + \mathbf{B} \mathbf{u}_0^{\text{LQR}}) \end{aligned}$$

$\mathbf{x}_1$

MPC

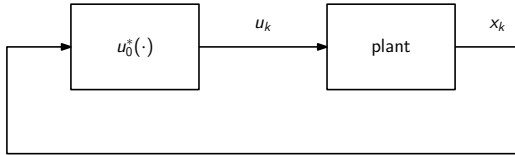
- ▶ Backward computation of $\mathbf{P}_K, \mathbf{P}_{K-1}, \dots, \mathbf{P}_1$
- ▶ Linear **time-invariant** feedback $\mathbf{u}_k = \mathbf{\Gamma}_0 \mathbf{x}_k$

$$\begin{aligned} \mathbf{u}_0^{\text{MPC}} &= -(\mathbf{R} + \mathbf{B}^\top \mathbf{P}_1 \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{P}_1 \mathbf{A} \mathbf{x}_0 = \mathbf{u}_0^{\text{LQR}} \\ \mathbf{u}_1^{\text{MPC}} &= -(\mathbf{R} + \mathbf{B}^\top \mathbf{P}_1 \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{P}_1 \mathbf{A} (\mathbf{A} \mathbf{x}_0 + \mathbf{B} \mathbf{u}_0^{\text{MPC}}) \end{aligned}$$

$\mathbf{x}_1$

same as LQR

different, LTI feedback



Option 1: Compute the **function** $u_0^*(\cdot)$ offline

- ▶ extremely complex with the exception of very few cases
- ▶ abundant offline computation power
- ▶ large memory requirement

Option 2: Compute the **value** $u_0^*(\cdot)$ online at every iteration

- ▶ often a tractable optimization problem
- ▶ limited time to perform the computation online

convex optimization

compute u_0^ fast enough $\Rightarrow$ between timesteps*

Option 1: Explicit computation of u_0^*

We already know one case.

Unconstrained linear MPC

→ first element of the finite-time LQR solution → linear feedback $u_0^*(\cdot)$

How about constraints?

Linear MPC + linear constraints*

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{x}} \quad & \sum_{k=0}^{K-1} \left(\mathbf{x}_k^\top \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^\top \mathbf{R} \mathbf{u}_k \right) + \mathbf{x}_K^\top \mathbf{S} \mathbf{x}_K, \quad \mathbf{Q}, \mathbf{S} \succeq 0, \mathbf{R} \succ 0 \\ \text{subject to} \quad & \mathbf{x}_{k+1} = \underbrace{\mathbf{A} \mathbf{x}_k + \mathbf{B} \mathbf{u}_k}_{\text{polytope } \mathbf{C} \mathbf{x} \leq \mathbf{d}} \\ & \mathbf{x} \in \mathcal{P}_{\mathbf{x}} \quad (\text{polytope } \mathbf{C} \mathbf{x} \leq \mathbf{d}) \\ & \mathbf{u} \in \mathcal{P}_{\mathbf{u}} \quad (\text{polytope } \mathbf{E} \mathbf{u} \leq \mathbf{f}) \end{aligned}$$

Handwritten notes:

- $a \leq x \leq b \rightarrow x \leq b$
- $a \leq x$
- $-x \leq -a$

* Extremely common MPC problem formulation

$$\begin{bmatrix} \mathbf{I} \\ -\mathbf{I} \end{bmatrix} \mathbf{x} \leq \begin{bmatrix} \mathbf{b} \\ -\mathbf{a} \end{bmatrix}$$

Example

Consider the **toy problem** (time horizon 1, scalar integrator)

$$\begin{aligned} \min_{u_0, x_1} \quad & u_0^2 + x_1^2 \\ \text{subject to} \quad & \underline{x_1 = x_0 + u_0} \\ & \underline{x_1 \leq 1} \end{aligned}$$

$$x_1 - 1 \leq 0$$

- ▶ Quadratic problem with linear constraints
- ▶ Parametrized in x_0
- ▶ The equality constraint (model) allows to eliminate x_1
- ▶ Convex $\Rightarrow$ KKT conditions are necessary and sufficient*

KKT conditions (only inequalities)

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & g(x) \leq 0 \end{aligned}$$

$\rightarrow$

$$\begin{cases} \nabla f(x) + \mu^\top \nabla g(x) = 0, & \mu \geq 0 \\ g(x) \leq 0 \\ \mu_i \underbrace{g_i(x)} = 0 & \forall i \end{cases}$$

$$\begin{aligned}\underline{f(u_0)} &= u_0^2 + (x_0 + u_0)^2 \\ \underline{g(u_0)} &= x_1 - 1 = \underline{x_0 + u_0 - 1} \quad \bullet \\ \nabla f(u_0) &= \underline{4u_0 + 2x_0} = 0 \\ \underline{\nabla g(u_0)} &= 1\end{aligned}$$

Let's consider the complementary slackness condition $\mu g(u_0) = 0$.

Case 1: $\mu = 0$ (and $\underline{g(u_0) \leq 0}$)

We then have $\nabla f(u_0) = 0$, which means $\underline{u_0 = -x_0/2}$.

With this input $g(u_0)$ then becomes $\underline{x_0/2 - 1}$, so that $\underline{g(u_0) \leq 0}$ means $\underline{x_0 \leq 2}$.

If $x_0 \leq 2$ then $u_0^* = -x_0/2$.

Case 2: $\underline{g(u_0) = 0}$ (and $\underline{\mu \geq 0}$)

$g(u_0) = 0$ implies that $u_0 = 1 - x_0$.

Because $\underline{\mu \geq 0}$, we must have $\underline{\nabla f(u_0) \leq 0}$ which means $\underline{4u_0 + 2x_0 \leq 0}$.

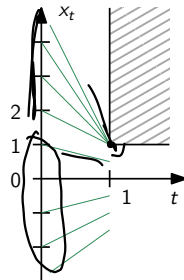
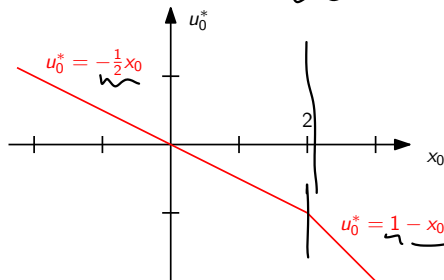
With $u_0 = 1 - x_0$ this means $\underline{4 - 2x_0 \leq 0}$, that is $\underline{x_0 \geq 2}$.

If $x_0 \geq 2$ then $u_0^* = 1 - x_0$.

Piecewise-affine feedback

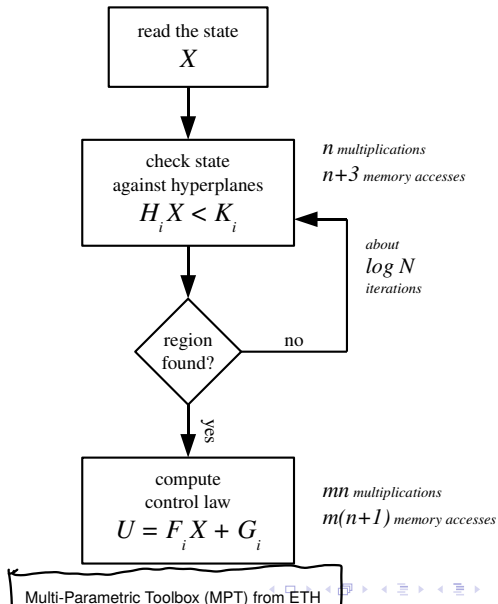
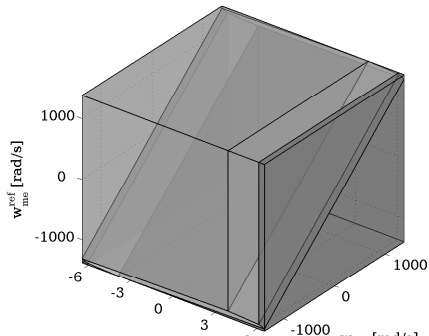
$$\begin{aligned} \min_{u_0, x_1} \quad & u_0^2 + x_1^2 \\ \text{subject to} \quad & x_1 = x_0 + u_0 \\ & x_1 \leq 1 \end{aligned}$$

- ▶ $u_0^*(\cdot)$ is continuous
- ▶ $u_0^*(\cdot)$ is piecewise affine
- ▶ The number of affine regions depends on the number of constraints (how?)
- ▶ In each region, the same constraints are active/inactive



Explicit MPC in practice

- ▶ Tens or hundreds of constraints
 - ▶ exponentially many regions
 - ▶ → very long **offline optimization**
 - ▶ even searching for the region is challenging in real-time!
 - ▶ → even longer **offline preprocessing** (binary search tree)
- 20 ms for $K = 5$, $n = 3$ (10 years ago)



Option 2: online computation



HOME > OFFERS > INDUSTRIAL SOFTWARE SOLUTIONS > MODEL-PREDICTIVE CONTROL - MPC

GLOBAL SITE ▾

Use cases

The typical use cases in the mining, minerals, cement, pulp and paper, oil and gas or marine sectors cover the vertical industries where ABB has a strong footprint.



Mining and minerals processing

Grinding and flotation - separation of valuable minerals from waste



Underground mining

Optimal mine ventilation with minimum power consumption



Terminals, stockyards at steel plants or mines

Stockyard Management



Cement production

Kilns, mills, alternative fuel management, material blending



Metals

Sintering and pelletizing process stability, quality and output



Aluminium

Improvement of thickness tolerances for a two-stand aluminium cold rolling mill



Pulp mills

Continuous digester of cellulose fiber or pulp



Paper mills

Wet End Control APC that stabilizes paper machine operations



Paper mills

Paper machine drives, winder, paper quality, sheet break performance, strength / weight virtual measurement



Industrial steam supply

Steam generation for industrial plants



Energy industries

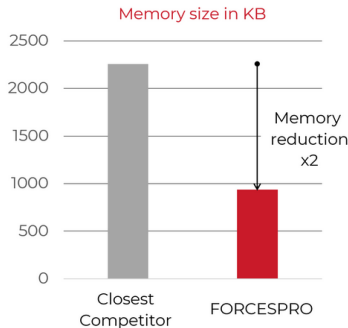
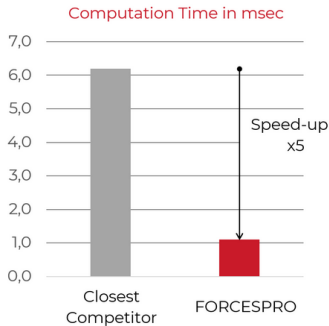
Distillation columns for crude oil



Marine

Vessel fuel management

BENCHMARK OF A NONLINEAR MPC OPTIMIZATION PROBLEM FOR A DC MOTOR WITH PREDICTION HORIZON N=105

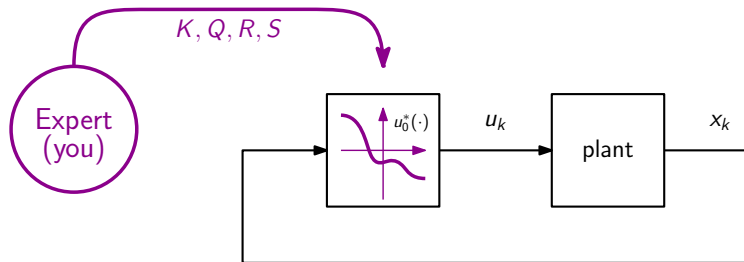


from www.embotech.com, ETH spin-off

Online MPC is increasingly common!

- ▶ custom optimization solvers
- ▶ approximate solutions of the optimization problem
- ▶ distributed solvers → distributed optimization
- ▶ faster hardware →

MPC as a feedback design tool



Ultimately, MPC is just a “clever” (principled) way to design static, time-invariant, non-linear feedback laws.

Closed-loop stability

- ▶ **Analysis:** compute u_0^* (hard!) and assess stability
- ▶ **Design:** derive conditions on K, Q, R, S that guarantee closed-loop stability

Tools:

Small-signal (local) stability?
Large-signal (global) stability?

Stable equilibrium

→ general

The equilibrium $x = 0$ is stable for the dynamics $x_{k+1} = f(x_k)$ if a small perturbation of the state perturbs the subsequent state trajectory in a continuous manner.

$$\underbrace{\forall \epsilon > 0, \exists \delta > 0 \text{ such that}} \underbrace{\|x_0\| < \delta \Rightarrow} \underbrace{\|x_k\| < \epsilon} \underbrace{\forall k \geq 0.}$$

Asymptotically stable equilibrium

The equilibrium $x = 0$ is asymptotically stable if it is stable and $\lim_{k \rightarrow \infty} \|x_k\| = 0$.

dissipativity

Lyapunov theorem

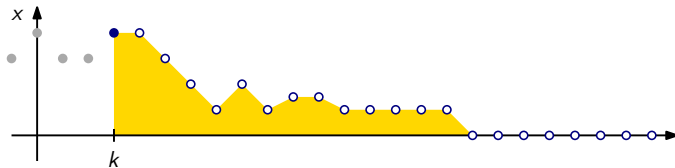
Let W be a real-valued function such that

$$\text{i f } \underbrace{W(0) = 0 \text{ and } W(x) > 0}_{\forall x \neq 0} \underbrace{W(f(x)) < W(x)}_{\forall x \neq 0}$$

Then $x = 0$ is asymptotically stable, under $x_{k+1} = f(x_k)$

→ <https://arxiv.org/abs/1809.05289>

Stability of infinite horizon MPC

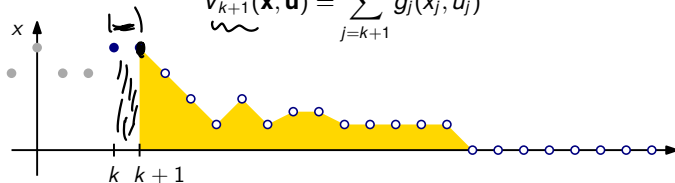


At time k , the MPC controller determines a trajectory $\mathbf{u}^*$ that minimizes

$$V_k^\infty(\mathbf{x}, \mathbf{u}) = \sum_{j=k}^{\infty} g_j(x_j, u_j)$$

By Bellman's principle, this trajectory minimizes also the "tail" cost

$$V_{k+1}^\infty(\mathbf{x}, \mathbf{u}) = \sum_{j=k+1}^{\infty} g_j(x_j, u_j)$$



This makes $W(x) = \min_{u,x} V_k^\infty$ an obvious candidate **Lyapunov function**, under very reasonable assumptions (for example on the functions g_s).

- ▶ it is zero only at the target equilibrium $x = 0$
- ▶ it decreases along the closed-loop trajectories of the system

$$\underbrace{W(x_{k+1})} = \underbrace{W(x_k)} - g_k(x_k, u_0^*(x_k)) < \underbrace{W(x_k)}$$

$\Rightarrow$ the equilibrium $x = 0$ is **asymptotically stable** for the closed-loop system.

This makes $W(x) = \min_{u,x} V_k^\infty$ an obvious candidate **Lyapunov function**, under very reasonable assumptions (for example on the functions g_s).

- ▶ it is zero only at the target equilibrium $x = 0$
- ▶ it decreases along the closed-loop trajectories of the system

$$W(x_{k+1}) = W(x_k) - g_k(x_k, u_0^*(x_k)) < W(x_k)$$

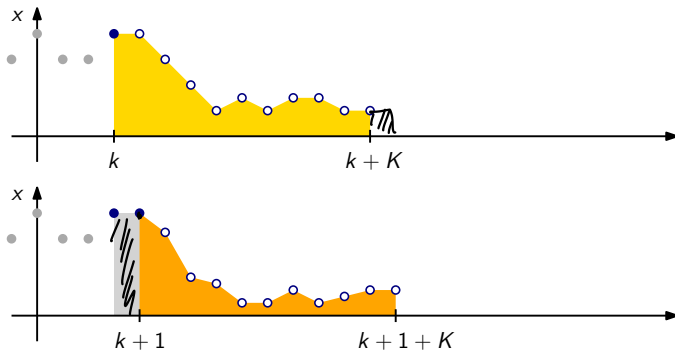
⇒ the equilibrium $x = 0$ is **asymptotically stable** for the closed-loop system.

Some subtleties

- ▶ Technically, we haven't defined the MPC control with an infinite-dimensional optimization problem
- ▶ We are assuming that the global minimum is found and attainable
- ▶ The functions g need to satisfy some properties (observability)

Finite-time MPC stability

The same trick clearly does not apply to finite-time receding-horizon MPC.

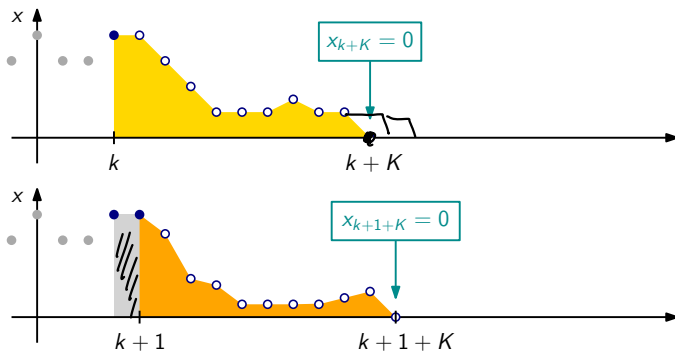


- ▶ At $k+1$, a different cost is minimized → different minimizer $\mathbf{x}, \mathbf{u}$
- ▶ The first element of the cost is removed, but a new element enters the tail.

Stability with terminal constraint

$$W(x) = \min_{u, x} \sum_{k=0}^{K-1} g_k(x_k, u_k)$$

subject to $x_{k+1} = f(x_k, u_k)$
 $x_0 = x, \underline{x_K = 0}$



Theorem

The equilibrium $x = 0$ is asymptotically stable for the closed-loop dynamics generated by a finite-time MPC controller with **zero terminal constraint**.

$$\underbrace{W(x_{k+1})}_{\text{value at } k+1} = \min \sum_{s=k+1}^{k+K} g(x_s, u_s) = \min \left(\sum_{s=k}^{k+K-1} g(x_s, u_s) - g(x_k, u_k) + g(x_{k+K}, u_{k+K}) \right)$$

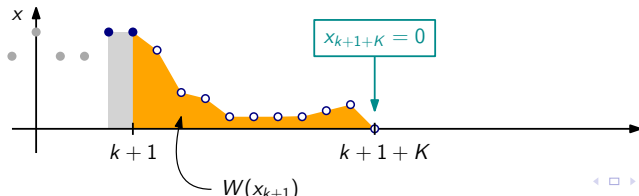
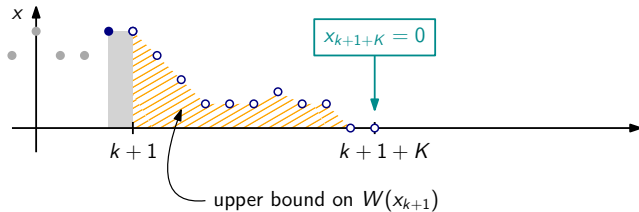
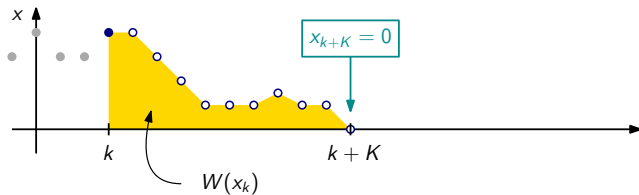
By definition of the minimum, this is small than the same quantity evaluated for an arbitrary u . Let's evaluate it for the input that minimizes the optimization problem at time k , that is

$$\tilde{u} = \arg \min \sum_{s=k}^{k+K-1} g(x_s, u_s),$$

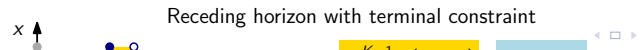
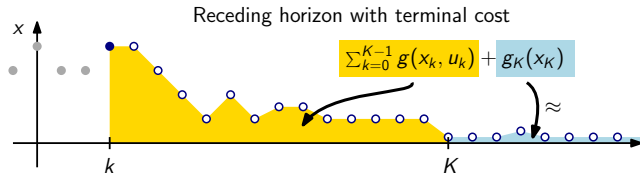
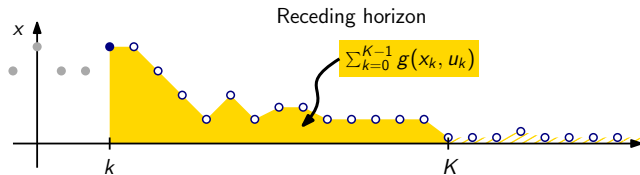
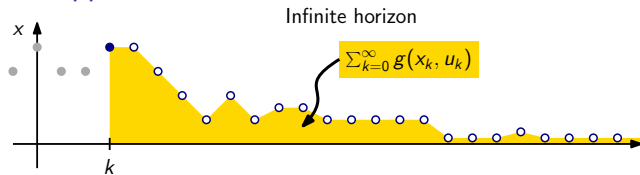
which yields $x_{k+K} = 0$, followed by $u_{k+K} = 0$ that maintains the systems at $x_{k+K+1} = 0$. We then have

$$W(x_{k+1}) \leq W(x_k) - g(\tilde{x}_k, \tilde{u}_k) \leq W(x_k).$$

Key proof idea



Receding horizon as an approximation of infinite horizon



MPC closed loop stability

Many alternative stability conditions have been derived

- ▶ K large enough
- ▶ zero terminal constraint
- ▶ terminal region for the state
- ▶ ...

MPC closed loop stability

Many alternative stability conditions have been derived

- ▶ K large enough
- ▶ zero terminal constraint
- ▶ terminal region for the state
- ▶ ...

Subtleties

- ▶ We always assumed feasibility of the MPC problem, which may not be true (for example with a zero terminal constraint)
- ▶ Future feasibility of the problem may depends on the current control actions.
- ▶ We always assume that the global minimum can be found.

MPC minimizes a given cost function of the system state and input trajectories

MPC is just a clever way to design a static time-invariant feedback control law

MPC is a brute force numerical construction of Lyapunov functions



$$\begin{aligned} \min_{\mathbf{u}, \mathbf{x}} \quad & \sum_{t=0}^{T-1} \left(x_t^\top Q x_t + u_t^\top R u_t \right) + x_T^\top S x_T, \quad Q, S \succeq 0, R \succ 0 \\ \text{subject to} \quad & x_{t+1} = A x_t + B u_t \\ & \mathbf{x} \in \mathcal{X} \\ & \mathbf{u} \in \mathcal{U} \end{aligned}$$

We have implicitly assumed that we are trying to regulate the steady state

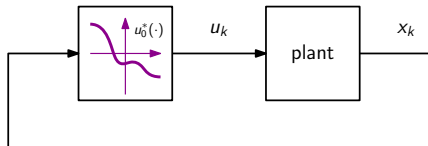
$$\underline{x_s = 0} \quad u_s = 0$$

However

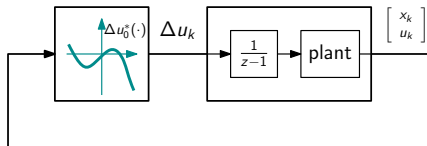
- ▶ we usually want plants to operate at a desirable working point x_s
(not to stop! $x_s = 0$)
- ▶ we can often allow a constant input u_s

Incremental formulation of MPC

Remember: MPC is “just” a well-designed non-linear static feedback law.



- ▶ If a non-zero input is necessary to drive the system to zero, then MPC will **not** achieve $x = 0$.
- ▶ Particularly fragile against
 - ▶ input disturbances
 - ▶ process noise
 - ▶ multiplicative noise uncertainty
- ▶ Very low input weights may cause numerical issues.



- ▶ $u_{k+1} = u_k + \Delta u_k$
- ▶ augment the plant with input integrator (if not present already!)
- ▶ integral nonlinear feedback law
- ▶ input weight penalizes changes in input signal

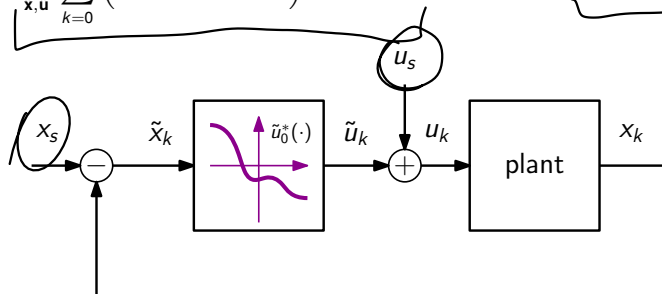
Non-zero steady state

If a desired equilibrium (x_s, u_s) is known, MPC can be simply be used to stabilize the system around that point.

change of coordinates $\begin{cases} \tilde{x} = x - \underline{x_s} \\ \tilde{u} = u - \underline{u_s} \end{cases}$

For linear systems, the change of variables is **inconsequential**.

$$V(x_0) = \min_{\tilde{x}, \tilde{u}} \sum_{k=0}^{K-1} (\tilde{x}_k^\top Q \tilde{x}_k + \tilde{u}_k^\top R \tilde{u}_k) + \tilde{x}_K^\top S \tilde{x}_K \quad \text{subject to} \quad \tilde{x}_{k+1} = A\tilde{x}_k + B\tilde{u}_k$$

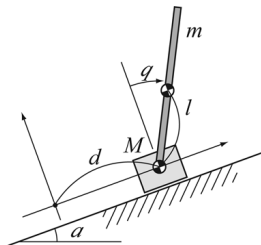


Steady-state selection

How do you decide the steady state (equilibrium) (x_s, u_s) ?

In some cases, it can be derived from first principles.

$$q = a, \quad \dot{q} = 0, \quad d = D, \quad \dot{d} = 0, \quad u = g \sin a$$



In other cases, specifications indicate a “desired” steady state $(x_{\text{spec}}, u_{\text{spec}})$.

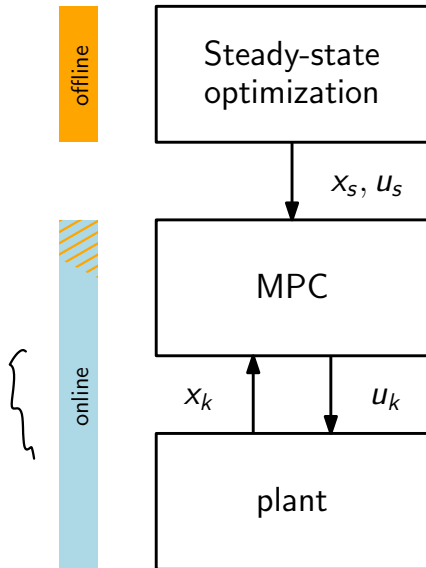
Steady-state selection problem

$$\begin{aligned} & \min_{x_s, u_s} \|x_s - x_{\text{spec}}\|_{Q_s}^2 + \|u_s - u_{\text{spec}}\|_{R_s}^2 \\ & \text{subject to } \underbrace{\begin{bmatrix} I - A & -B \end{bmatrix}}_{\substack{x_s \in \mathcal{X} \\ u_s \in \mathcal{U}}} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = 0 \quad \text{(a valid steady state)} \end{aligned}$$

$$x_{n+1} = \underbrace{Ax_n + Bu_n}_{= x_n}$$

$$(A - I)x_h + Bu_h = 0$$

MPC control architecture



~~More on this later...~~

Not surprisingly, MPC relies on having access to a **model** of the system.

- ▶ Model mismatch
 - ▶ what is the effect of an **approximate model** on the feedback gain?
 - ▶ is (x_s, u_s) a steady-state? $\rightarrow$ extremely sensitive
- ▶ Output/measurement noise
 - ▶ what is the effect of an **additive noise to the measurement**?
- ▶ Process noise
 - ▶ what is the effect of an **additive noise to the input**?

} Kalman Filter

$$x_{k+1} = Ax_k + Bu_k + Bd_k$$

$$y_k = Cx_k + Cd_k$$

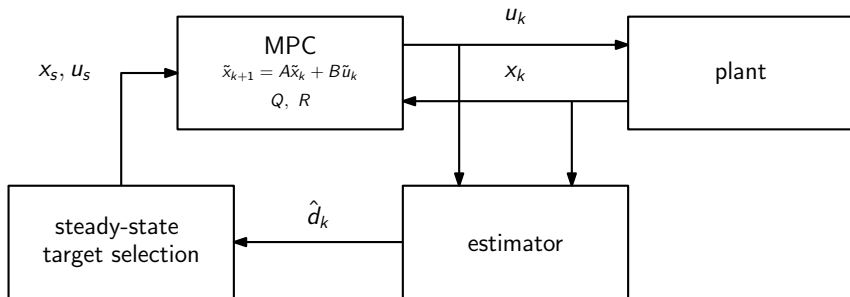
Desired behavior (without invoking tools like robust MPC)

In the presence of any of these disturbances, either

- ▶ zero steady-state offset, or
- ▶ unbounded trajectory (instability), or
- ▶ active constraints.

Disturbance rejection in MPC

1. Add a **model for the disturbance**
2. Based on the augmented model and the available measurements, **estimate the disturbance**
3. Correct the **steady-state target** based on the estimate of the disturbance



Step 1: Add a model for the disturbance

Unless better information is available, a constant model is appropriate

$$d_{k+1} = d_k \quad d_k \in \mathbb{R}^{n_d}, \quad n_d \leq n$$

Good regulator theorem

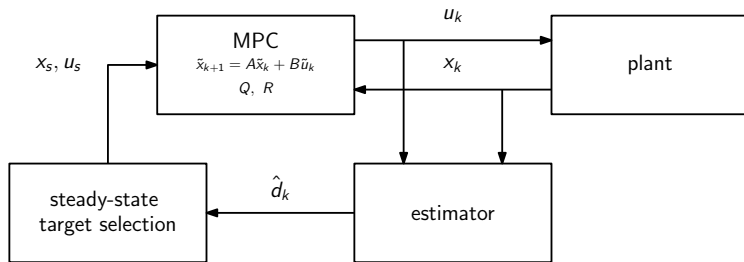
“Every good regulator of a system must be a model of that system”

If a better model is available, use it!

- ▶ periodic disturbance? ramps?
- ▶ see Internal Model Principle
- ▶ the closed loop system will reject disturbances of the class that you modeled
- ▶ how many integrators do you need to reject a step disturbance? a ramp?

$$\begin{bmatrix} x_{k+1} \\ d_{k+1} \end{bmatrix} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix} \begin{bmatrix} x_k \\ d_k \end{bmatrix} + \begin{bmatrix} B \\ 0 \end{bmatrix} u_k$$

Step 2: Estimate the disturbance



For example, a Luenberger state estimator

- **prediction** of x_{k+1} based on **measurements** x_k, u_k and **estimate** $\hat{d}_k$

$$\hat{x}_{k+1} = Ax_k + Bu_k + B_d \hat{d}_k$$

- **correction** based on the **prediction error**

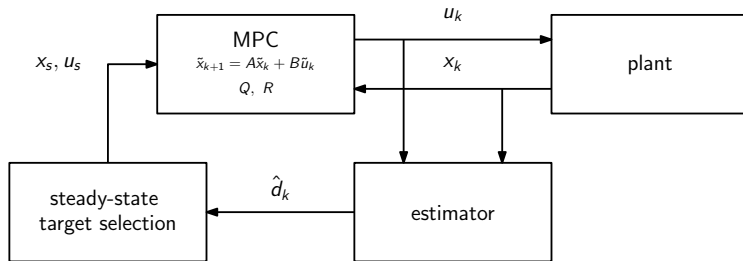
$$\hat{d}_{k+1} = \hat{d}_k + L(x_k - \hat{x}_k)$$

Correct the steady-state target

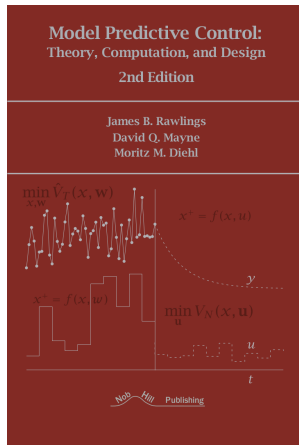
Based on the constant disturbance model, $\hat{d}_k$ is our best estimate of the steady-state d_s .

Modified steady-state selection problem

$$\begin{aligned} \min_{x_s, u_s} \quad & \|x_s - x_{\text{spec}}\|_{Q_s}^2 + \|u_s - u_{\text{spec}}\|_{R_s}^2 \\ \text{subject to} \quad & \begin{bmatrix} I - A & -B \end{bmatrix} \begin{bmatrix} x_s \\ u_s \end{bmatrix} = B_d \hat{d}_k \\ & x_s \in \mathcal{X} \\ & u_s \in \mathcal{U} \end{aligned}$$



And more...



- ▶ Output-feedback MPC
- ▶ Terminal constraints
- ▶ More powerful stability guarantees
- ▶ Examples

Chapters 1, 2, 7 of

Model Predictive Control:
Theory, Computation, and Design

James B. Rawlings, David Q. Mayne, Moritz M. Diehl
2nd edition, 2022

Free to download

<https://sites.engineering.ucsb.edu/~jbrow/mpc/>

Data-Enabled Predictive Control

System response

$$y_t = \underbrace{CA^t x_0}_{\text{free response}} + \underbrace{\sum_{j=0}^{t-1} CA^{t-1-j} Bu_j}_{\text{convolution}}$$

$$x_{n+1} = Ax_n + Bu_n$$

$$y_n = Cx_n$$

$$y_0 = Cx_0$$

$$y_1 = Cx_{n+1} = C(Ax_0 + Bu_0)$$
$$\vdots$$

Simplifying assumption (SISO)

For now, assume a scalar input $u \in \mathbb{R}$ and a scalar output $y \in \mathbb{R}$.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{L-1} \end{bmatrix} = \underbrace{\begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{L-1} \end{bmatrix}}_{\mathcal{O}_L} x_0 + \underbrace{\begin{bmatrix} 0 & 0 & \dots & 0 & 0 & \dots \\ CB & 0 & \dots & CB & 0 & \dots \\ CAB & CB & 0 & \dots & \dots & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ CA^{L-2}B & CA^{L-3}B & \dots & CAB & CB & 0 \end{bmatrix}}_{\mathcal{G}_L} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{L-1} \end{bmatrix}$$

- ▶ $\mathcal{O}_L \in \mathbb{R}^{L \times n}$ (tall) is the **extended observability matrix**
- ▶ $\mathcal{G}_L \in \mathbb{R}^{L \times L}$ (square) is the **convolution matrix**

Free response

$$x_n \in \mathbb{R}^n$$

$$\underbrace{\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{L-1} \end{bmatrix}}_{\underline{y_L}} = \underbrace{\begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{L-1} \end{bmatrix}}_{\mathcal{O}_L} \underbrace{x_0}_{\text{wavy line}}$$

Assumption: Observable system (SISO)

$\mathcal{O}_n$, with n the state dimension, is full rank.

Last week we focused on whether we can reconstruct x_0 by looking at the output sequence.

- ▶ If $L < n$, then x_0 cannot be reconstructed (not enough information)
- ▶ If $L = n$, then x_0 can be reconstructed: $x_0 = \mathcal{O}_n^{-1} \underline{y_n}$
- ▶ If $L > n$, then the question is overdetermined:
 - ▶ x_0 can be reconstructed from a valid output sequence
 - ▶ not all y sequences are valid!

Example, $n = 1$

Homework

$$x_{t+1} = \frac{1}{2}x_t + u_t$$

$$y_t = x_t$$

$$\mathcal{O}_L = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{L-1} \end{bmatrix} = \begin{bmatrix} 1 \\ 1/2 \\ 1/4 \\ \vdots \\ 1/2^{L-1} \end{bmatrix}$$

Assuming no input, can you reconstruct x_0 from

- ▶ $y_0 = 1$?
- ▶ $y_0 = 2, y_1 = 1$?
- ▶ $y_0 = 1, y_1 = 1$?

What are all the output sequences produced by the free response of this system?

Exercise: Try with an undamped oscillator, $n = 2$.



Suppose that the input sequence $u_0, \dots, u_L$ is non zero, but **known**.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{L-1} \end{bmatrix} = \underbrace{\begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{L-1} \end{bmatrix}}_{\mathcal{O}_L} x_0 + \underbrace{\begin{bmatrix} 0 & 0 & \dots & & \\ CB & 0 & \dots & & \\ CAB & CB & 0 & \dots & \\ \vdots & & & & \\ CA^{L-2}B & CA^{L-1}B & \dots & CAB & CB & 0 \end{bmatrix}}_{\mathcal{G}_L} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{L-1} \end{bmatrix}$$

Can you determine x_0 ?

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{L-1} \end{bmatrix} - \underbrace{\begin{bmatrix} 0 & 0 & \dots & & \\ CB & 0 & \dots & & \\ CAB & CB & 0 & \dots & \\ \vdots & & & & \\ CA^{L-2}B & CA^{L-1}B & \dots & CAB & CB & 0 \end{bmatrix}}_{\mathcal{G}_L} \underbrace{\begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_{L-1} \end{bmatrix}}_{\text{known}} = \underbrace{\begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{L-1} \end{bmatrix}}_{\mathcal{O}_L} x_0$$

$$\mathbf{y}_L - \mathcal{G}_L \mathbf{u}_L = \mathcal{O}_L x_0$$

Similar to before, the problem can be solved if

$$\mathbf{y}_L - \mathcal{G}_L \mathbf{u}_L$$

is in the column image of $\mathcal{O}_L$.

For scalar systems

- ▶ if $L < n$, underdetermined
- ▶ if $L = n$, then $x_0 = \mathcal{O}_n^{-1} (\mathbf{y}_n - \mathcal{G}_n \mathbf{u}_n)$
- ▶ if $L > n$, overdetermined

Admissible input-output sequences

Interesting consequence: for $L > n$, $\mathbf{y}_L - \mathcal{G}_L \mathbf{u}_L$ needs to belong to a subspace!

All input-output trajectories of length N of the system need to satisfy

$$\begin{bmatrix} \mathbf{u}_L \\ \mathbf{y}_L \end{bmatrix} = \underbrace{\begin{bmatrix} I_L & 0 \\ \mathcal{G}_L & \mathcal{O}_L \end{bmatrix}}_{\Lambda_L} \begin{bmatrix} \mathbf{u}_L \\ x_0 \end{bmatrix}$$

Handwritten notes: $\mathbf{u}_L$ is circled and labeled "know". Λ_L is labeled "construct from (A,B)".

- ▶ $\Lambda_L \in \mathbb{R}^{(L+L) \times (L+n)}$
- ▶ if $L \geq n$, Λ_L has rank $L + n$

Behavioral representation

Trajectories $\mathbf{u}_L, \mathbf{y}_L$ of the system need to belong to the **column image** of Λ_L .

- ▶ Λ_L is a full description of the system, exactly like A, B, C
- ▶ If you are given the matrix Λ_L , you can tell whether an input/output sequence is compatible with that plant
 - ▶ compatible = "there exists an initial state x_0 such that..."

"Behavioral system theory" by Jan C. Willems, from approx the 80s.
System theory without classifying signals as inputs and outputs!

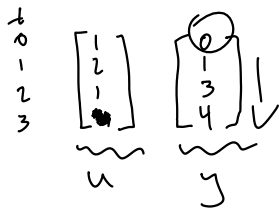
$L = n$ (observability problem)

$$\begin{bmatrix} \mathbf{u}_n \\ \mathbf{y}_n \end{bmatrix} = \underbrace{\begin{bmatrix} I_n & 0 \\ \mathcal{G}_n & \mathcal{O}_n \end{bmatrix}}_{\Lambda_n} \begin{bmatrix} \mathbf{u}_n \\ x_0 \end{bmatrix}$$

- ▶ Λ_n has dimension $2n \times 2n$
- ▶ Λ_n has rank $2n$
- invertible matrix

Any input/output sequence of length n can be “explained” by an initial condition x_0 .

Toy example:



$$\begin{aligned} x_{t+1} &= x_t + u_t \\ \underline{y_t} &= \underline{x_t} \end{aligned}$$

$$L = n + 1$$

$$\left[\begin{array}{c} \left[\begin{array}{c} u_0 \\ \vdots \\ u_{n-1} \\ u_n \end{array} \right] \\ \left[\begin{array}{c} y_0 \\ \vdots \\ y_{n-1} \\ y_n \end{array} \right] \end{array} \right] = \underbrace{\begin{bmatrix} I_{n+1} & 0 \\ \mathcal{G}_{n+1} & \mathcal{O}_{n+1} \end{bmatrix}}_{\Lambda_{n+1}} \left[\begin{array}{c} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ x_0 \end{array} \right]$$

- ▶ Λ_{n+1} has dimension $\underline{2(n+1)} \times \underline{2n+1}$
- ▶ Λ_{n+1} has rank $\underline{L+n} = \underline{2n+1}$
- ▶ the input/output sequence is a vector in $\underline{\mathbb{R}^{2(n+1)}}$
- the input/output sequence belongs to a subspace of dimension $2n+1$

Toy example:

$L = n + 1$ (prediction problem)

$$\begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ y_0 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix} = \underbrace{\begin{bmatrix} I_{n+1} & 0 \\ \mathcal{G}_{n+1} & \mathcal{O}_{n+1} \end{bmatrix}}_{\Lambda_{n+1}} \begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ \cancel{x_0} \end{bmatrix}$$

Prediction problem

Suppose that y_n is missing. Can you reconstruct it?

► Intuitively:

- We already know that we can reconstruct x_0 from the first n input/output samples
- Once we have x_0 we can “simulate” the output until y_n .

$L = n + K$ (prediction problem)

$$\left[\begin{array}{c} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ \vdots \\ u_{n+K-1} \\ y_0 \\ \vdots \\ y_{n-1} \\ y_n \\ \vdots \\ y_{n+K-1} \end{array} \right] = \underbrace{\begin{bmatrix} I_{n+K} & 0 \\ \mathcal{G}_{n+K} & \mathcal{O}_{n+K} \end{bmatrix}}_{\Lambda_{n+K}} \left[\begin{array}{c} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ \vdots \\ u_{n+K-1} \\ x_0 \end{array} \right]$$

Prediction problem

Suppose that the sequence $y_n \dots y_{n+K-1}$ is missing. Can you reconstruct it?

“Linear-algebra” interpretation

Any I/O sequence of length $L > n$ belongs to a subspace of dimension $L + n$

$$\begin{bmatrix} u_0 \\ \vdots \\ u_{n-1} \\ u_n \\ \vdots \\ u_{n+K-1} \\ y_0 \\ \vdots \\ y_{n-1} \\ y_n \\ \vdots \\ y_{n+K-1} \end{bmatrix} = \underbrace{Hg}_{\omega}$$

Image representation

- ▶ columns of $H \in \mathbb{R}^{2L \times (L+n)}$ are a basis of the subspace
- ▶ $g \in \mathbb{R}^{L+n}$ is a vector of coefficients

- ▶ A basis for this subspace is given by the columns on Λ_L
 - ▶ We need A, B, C to build Λ_L
- ▶ Can you find another basis?

- ▶ A basis for this subspace is given by the columns on Λ_L
 - ▶ We need A, B, C to build Λ_L
- ▶ Can you find another basis?

Hint: m linearly independent vectors from a subspace of dim. m form a basis.

How would you generate columns from this subspace?

- ▶ A basis for this subspace is given by the columns on Λ_L
 - ▶ We need A, B, C to build Λ_L
- ▶ Can you find another basis?

Hint: m linearly independent vectors from a subspace of dim. m form a basis.

How would you generate columns from this subspace? Data!

$$\begin{bmatrix} \mathbf{u}_L^{(1)} \\ \mathbf{y}_L^{(1)} \end{bmatrix} \begin{bmatrix} \mathbf{u}_L^{(2)} \\ \mathbf{y}_L^{(2)} \end{bmatrix} \begin{bmatrix} \mathbf{u}_L^{(3)} \\ \mathbf{y}_L^{(3)} \end{bmatrix} \cdots \begin{bmatrix} \mathbf{u}_L^{(L+n)} \\ \mathbf{y}_L^{(L+n)} \end{bmatrix}$$

- ▶ A basis for this subspace is given by the columns on Λ_L
 - ▶ We need A, B, C to build Λ_L
- ▶ Can you find another basis?

Hint: m linearly independent vectors from a subspace of dim. m form a basis.

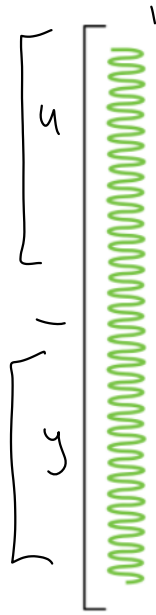
How would you generate columns from this subspace? Data!

$$\begin{bmatrix} \mathbf{u}_L^{(1)} \\ \mathbf{y}_L^{(1)} \end{bmatrix} \begin{bmatrix} \mathbf{u}_L^{(2)} \\ \mathbf{y}_L^{(2)} \end{bmatrix} \begin{bmatrix} \mathbf{u}_L^{(3)} \\ \mathbf{y}_L^{(3)} \end{bmatrix} \cdots \begin{bmatrix} \mathbf{u}_L^{(L+n)} \\ \mathbf{y}_L^{(L+n)} \end{bmatrix}$$

- ▶ I/O sequences must belong to the subspace of valid signals
- ▶ **Assuming linear independence**, $L + n$ of them will form a basis.
- ▶ **All** and **only** the valid I/O sequences are of the form

$$\underbrace{\begin{bmatrix} \mathbf{u}_L \\ \mathbf{y}_L \end{bmatrix}} = \underbrace{\begin{bmatrix} \mathbf{u}_L^{(1)} & \mathbf{u}_L^{(2)} & \mathbf{u}_L^{(3)} & \cdots & \mathbf{u}_L^{(L+n)} \\ \mathbf{y}_L^{(1)} & \mathbf{y}_L^{(2)} & \mathbf{y}_L^{(3)} & \cdots & \mathbf{y}_L^{(L+n)} \end{bmatrix}}_H g$$

with $g \in \mathbb{R}^{L+n}$.



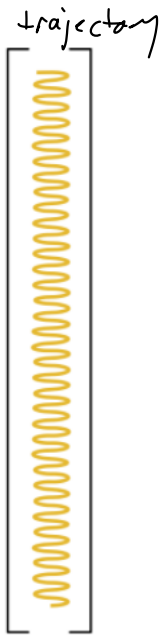
Time series data



Combination



$=$



Exercise

- ▶ Consider the scalar ($n = 1$) SISO system

$$x_{t+1} = x_t + u_t$$

$$y_t = x_t$$

- ▶ Consider input/output trajectories of length $L = 3$.
- ▶ Generate trajectories of length L (random initial conditions, random inputs)
- ▶ Check the rank of the matrix H for an increasing number of columns
- ▶ What is the maximum rank?

Data matrix

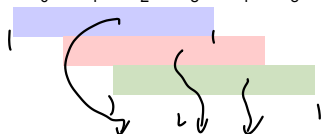
Data is expensive and hard to obtain.

- ▶ to predict K steps in the future for a system of dimension n we need trajectories of length $L = K + n$
- ▶ we need at least $L + n = K + 2n$ of them

A very efficient use of your data points:

very long

... u_0 u_1 u_2 u_3 u_4 u_5 u_6 u_7 ...



rearranged as

$$\mathcal{H}_L(\mathbf{u}) = \begin{bmatrix} u_0 & u_1 & u_2 & \dots & u_{L+n-1} \\ u_1 & u_2 & u_3 & & \\ u_2 & u_3 & u_4 & & \\ \dots & & & & \\ u_{L-1} & & & & u_{2L+n-2} \end{bmatrix}$$

Have you met this matrix before? $\rightarrow$ Hankel matrix

Persistence of Excitation
 $\Rightarrow \mathcal{H}_L(\mathbf{u})$ has full rank

$$\mathcal{H}_L(\mathbf{u}) = \begin{bmatrix} u_0 & u_1 & u_2 & \dots & u_{L+n-1} \\ u_1 & u_2 & u_3 & & \\ u_2 & u_3 & u_4 & & \\ \dots & & & & \\ u_{L-1} & & & & u_{2L+n-2} \end{bmatrix}$$

$$\mathcal{H}_L(\mathbf{y}) = \begin{bmatrix} y_0 & y_1 & y_2 & \dots & y_{L+n-1} \\ y_1 & y_2 & y_3 & & \\ y_2 & y_3 & y_4 & & \\ \dots & & & & \\ y_{L-1} & & & & y_{2L+n-2} \end{bmatrix}$$

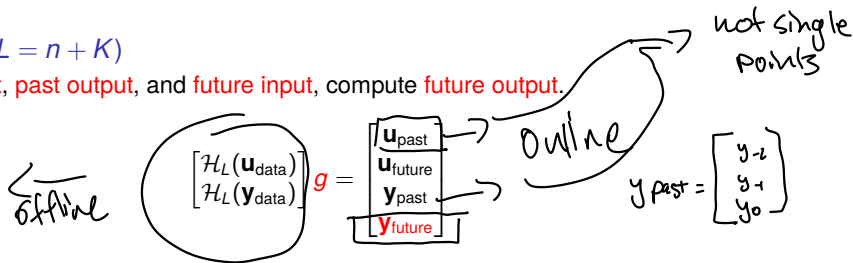
All system trajectories of length L are generated as

$$\begin{bmatrix} \mathcal{H}_L(\mathbf{u}) \\ \mathcal{H}_L(\mathbf{y}) \end{bmatrix} g$$

- ▶ $\mathbf{u}$ and $\mathbf{y}$ sequences of length $2L + n - 1$
- ▶ $g \in \mathbb{R}^{L+n}$ free vector of parameters

K -step predictor ($L = n + K$)

Given the **past input**, **past output**, and **future input**, compute **future output**.



- ▶ $\mathbf{g} \in \mathbb{R}^{2n+K}$ and $\mathbf{y}_{\text{future}} \in \mathbb{R}^K$ unknowns $\rightarrow$ total $2(n + K)$
- ▶ $2(n + K)$ equations

knowns

$\mathcal{H}_L(\mathbf{u}), \mathcal{H}_L(\mathbf{y})$

$\mathbf{u}_{\text{past}}, \mathbf{y}_{\text{past}}$

unknowns

$\mathbf{g}$

$\mathbf{u}_{\text{future}}$

$\mathbf{y}_{\text{future}}$

Many uses:

- ▶ Filtering
- ▶ Recover missing data
- ▶ Prediction / simulation
- ▶ **Control**

Model-based Predictive Control

Consider an **output tracking MPC** problem (SISO, for now).

At each time, the state x is measured and the following optimization problem is solved:

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{x}, \mathbf{y}} \quad & \sum_{k=0}^{K-1} y_k^2 + r u_k^2 \\ \text{subject to} \quad & \mathbf{x}_{k+1} = \mathbf{A} \mathbf{x}_k + \mathbf{B} \mathbf{u}_k \\ & y_k = \mathbf{C} \mathbf{x}_k \\ & \mathbf{x}_0 = \mathbf{x} \\ & \mathbf{u}_k \in \mathcal{U}_k \quad \forall k \\ & y_k \in \mathcal{Y}_k \quad \forall k \end{aligned}$$

The **model equations** enforce that $\mathbf{u}$ and $\mathbf{y}$ are trajectories of the plant for the next K steps.

Data-enabled Predictive Control

Replace the **state-space model** with our new **behavioral model**

once g is fixed, (say, g^a)

$$\begin{bmatrix} u_{\text{past}} \\ u \end{bmatrix} = A(u) g^a$$

$$\begin{bmatrix} y_{\text{past}} \\ y \end{bmatrix} = A(y) g^a$$

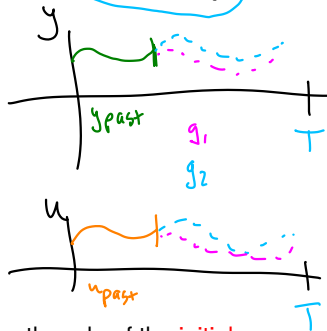
$$\min_{u, g} \sum_{k=0}^{K-1} y_k^2 + r u_k^2$$

subject to

$$\underbrace{\begin{bmatrix} \mathcal{H}_L(\mathbf{u}_{\text{data}}) \\ \mathcal{H}_L(\mathbf{y}_{\text{data}}) \end{bmatrix}}_{\substack{u_k \in \mathcal{U}_k \forall k \\ y_k \in \mathcal{Y}_k \forall k}} g = \begin{bmatrix} u_{\text{past}} \\ u \\ y_{\text{past}} \\ y \end{bmatrix}$$

$$A(u)g = \begin{bmatrix} u_{\text{past}} \\ u \end{bmatrix}$$

$$A(y)g = \begin{bmatrix} y_{\text{past}} \\ y \end{bmatrix}$$



where

- ▶ a new decision variable g has been introduced
- ▶ $\mathbf{u}_{\text{past}}$ and $\mathbf{y}_{\text{past}}$ are the last n inputs and outputs of the system, and play the role of the **initial condition**

$$\min_{\mathbf{u}, \mathbf{x}, \mathbf{y}} \sum_{k=0}^{K-1} y_k^2 + r u_k^2$$

subject to $\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}u_k$

$$y_k = \mathbf{C}\mathbf{x}_k$$

$$\mathbf{x}_0 = \mathbf{x}$$

$$u_k \in \mathcal{U}_k \quad \forall k$$

$$y_k \in \mathcal{Y}_k \quad \forall k$$

$$\min_{\mathbf{u}, \mathbf{y}, \mathbf{g}} \sum_{k=0}^{K-1} y_k^2 + r u_k^2$$

subject to

$$\begin{bmatrix} \mathcal{H}_L(\mathbf{u}_{\text{data}}) \\ \mathcal{H}_L(\mathbf{y}_{\text{data}}) \end{bmatrix} \mathbf{g} = \begin{bmatrix} \mathbf{u}_{\text{past}} \\ \mathbf{u} \\ \mathbf{y}_{\text{past}} \\ \mathbf{y} \end{bmatrix}$$

$$u_k \in \mathcal{U}_k \quad \forall k$$

$$y_k \in \mathcal{Y}_k \quad \forall k$$

- ▶ Computationally more complex ($2n + K$ additional decision variables)
- ▶ Larger memory footprint (data instead of parametric model $\mathbf{A}, \mathbf{B}, \mathbf{C}$)
- ▶ No additional state estimator needed.

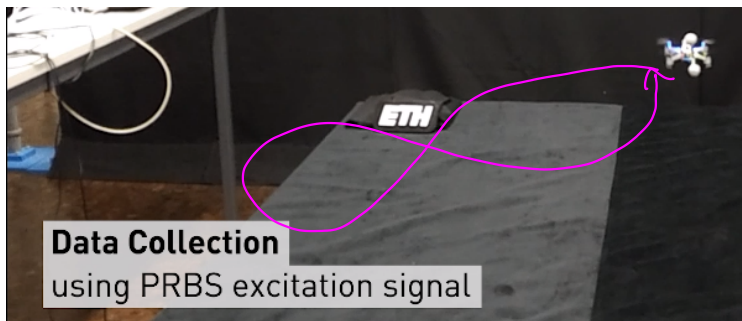
Assumption

We assumed **persistence of excitation** in the collected data

- ▶ technically, to ensure that the data matrix has full rank
- ▶ intuitively, because we want the data to represent all dynamics of the system

This is a standard requirement in **identification**, and it is typically resolved by

- ▶ noisy/random identification input sequences
- ▶ relying on ambient/process noise (for some applications)
- ▶ using historical data + a posteriori linear independence test



MIMO case

The theory extends very naturally to **multiple inputs** and **multiple outputs**.

Lag of a system

The lag of a system is the smallest ℓ such that

$$\mathcal{O}_\ell = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{\ell-1} \end{bmatrix}$$

has rank n .

- ▶ A past trajectory of length $T_{\text{ini}} \geq \ell$ is needed as “initial conditions”.
- ▶ For MIMO systems
 - ▶ input $u \in \mathbb{R}^m$
 - ▶ output $y \in \mathbb{R}^p$
 - ▶ state $x \in \mathbb{R}^n$

an input/output sequences of at length $T \geq (m+1)(T_{\text{ini}} + K) + n - 1$ is needed

- ▶ For SISO systems, $\ell = n$.

length of the
past trajectory
needed to set
IC

Lag and system size estimation

- a past trajectory of length $T_{\text{ini}} \geq \ell$ is needed as “initial conditions”
- input/output sequences of at length $T \geq (m + 1)(T_{\text{ini}} + K) + n - 1$ is needed

As $\ell \leq n$, in practice we need an a-priori estimate of the minimal realization size n .

- ▶ First principles
 - ▶ for example, electro-mechanical systems
- ▶ A posteriori verification
 - ▶ Build increasingly bigger Hankel matrices and verify the rank

Once the order is estimated, $T_{\text{ini}} = n$ is a valid solution, although in some cases side information can allow to choose a smaller T_{ini} .

K is determined by the control specifications (similar to MPC).

In theory, adding data to our data matrix does not hurt

- ▶ it ensures that we have enough data in case we are uncertain about n
- ▶ it ensures that all the system modes have been excited
- ▶ the rank of H stops increasing after $mL + n$

$$\underbrace{\begin{bmatrix} \mathbf{u}_L^{(1)} & \mathbf{u}_L^{(2)} & \mathbf{u}_L^{(3)} & \dots \\ \mathbf{y}_L^{(1)} & \mathbf{y}_L^{(2)} & \mathbf{y}_L^{(3)} & \dots \end{bmatrix}}_H \mathbf{g}$$

This is true only in the **noiseless case**.

In the presence of **noise in the data**, the rank of H will increase past $mL + n$.

Consequences of noise in the data

Consider the (pathological) case in which one of the outputs is identically zero.

$$y_k = \underline{C}x_k = \begin{bmatrix} \star & \star & \star \\ 0 & 0 & 0 \end{bmatrix} x_k$$

All output sequences should be of the form

$$y_k = \begin{bmatrix} \star \\ 0 \end{bmatrix}$$

However, if the data contains noise, some columns in the data matrix may present a non-zero second coordinate in the output

$$\underbrace{\begin{bmatrix} \mathbf{u}_L^{(1)} & \mathbf{u}_L^{(2)} & \mathbf{u}_L^{(3)} & \dots \\ \mathbf{y}_L^{(1)} & \mathbf{y}_L^{(2)} & \mathbf{y}_L^{(3)} + \epsilon & \dots \end{bmatrix}}_H \quad g$$

The controller is led to

- ▶ “assume” that the input sequence $\mathbf{u}_L^{(3)}$ is capable of affecting the last coordinate of the output!
- ▶ use a large g ($\rightarrow$ large input!) to try to affect the uncontrollable output

Enforcing the rank condition

Model selection and enforcement of model dimension was easier in **identification**.

Remember?

$$\mathcal{H}_{k,\ell} = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0_{r \times n-r} \\ 0_{m-r \times r} & 0_{m-r \times n-r} \end{bmatrix} \begin{bmatrix} V_1 & V_2 \end{bmatrix}^\top = U_1 \Sigma_1 V_1^\top$$

Suppose that we know that the system has dimension **exactly** n .

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{y}, g} \quad & \sum_{k=0}^{K-1} \|y_k\|_Q^2 + \|u_k\|_R^2 \\ \text{subject to} \quad & \underbrace{\begin{bmatrix} \mathcal{H}_L(\mathbf{u}_{\text{data}}) \\ \mathcal{H}_L(\mathbf{y}_{\text{data}}) \end{bmatrix}}_{\mathcal{H}} g = \begin{bmatrix} \mathbf{u}_{\text{past}} \\ \mathbf{u} \\ \mathbf{y}_{\text{past}} \\ \mathbf{y} \end{bmatrix} \\ & u_k \in \mathcal{U}_k \quad \forall k \\ & y_k \in \mathcal{Y}_k \quad \forall k \end{aligned}$$

H should have rank $L + n$ but most likely it won't.

Rank constrained data-driven control

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{y}, g, \hat{\mathbf{u}}_{\text{data}}, \hat{\mathbf{y}}_{\text{data}}} \quad & \sum_{k=0}^{K-1} \|\mathbf{y}_k\|_Q^2 + \|\mathbf{u}_k\|_R^2 \\ \text{subject to} \quad & (\hat{\mathbf{u}}_{\text{data}}, \hat{\mathbf{y}}_{\text{data}}) = \operatorname{argmin} \|\hat{\mathbf{u}}_{\text{data}} - \mathbf{u}_{\text{data}}\|^2 + \|\hat{\mathbf{y}}_{\text{data}} - \mathbf{y}_{\text{data}}\|^2 \\ & \text{subject to} \quad \operatorname{rank} \left(\begin{bmatrix} \mathcal{H}_L(\hat{\mathbf{u}}_{\text{data}}) \\ \mathcal{H}_L(\hat{\mathbf{y}}_{\text{data}}) \end{bmatrix} \right) = L + n \end{aligned}$$

$$\begin{bmatrix} \mathcal{H}_L(\hat{\mathbf{u}}_{\text{data}}) \\ \mathcal{H}_L(\hat{\mathbf{y}}_{\text{data}}) \end{bmatrix} g = \begin{bmatrix} \mathbf{u}_{\text{past}} \\ \mathbf{u} \\ \mathbf{y}_{\text{past}} \\ \mathbf{y} \end{bmatrix}$$

$$\mathbf{u}_k \in \mathcal{U}_k \quad \forall k$$

$$\mathbf{y}_k \in \mathcal{Y}_k \quad \forall k$$

- ▶ nested identification problem
- ▶ non convex (rank constraint)
- ▶ computationally hard

Step 1: At most $L + n$ non-zero elements of g are needed

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{y}, g, \hat{\mathbf{u}}_{\text{data}}, \hat{\mathbf{y}}_{\text{data}}} \quad & \sum_{k=0}^{K-1} \|\mathbf{y}_k\|_Q^2 + \|\mathbf{u}_k\|_R^2 \\ \text{subject to} \quad & (\hat{\mathbf{u}}_{\text{data}}, \hat{\mathbf{y}}_{\text{data}}) = \operatorname{argmin} \|\hat{\mathbf{u}}_{\text{data}} - \mathbf{u}_{\text{data}}\|^2 + \|\hat{\mathbf{y}}_{\text{data}} - \mathbf{y}_{\text{data}}\|^2 \\ & \text{subject to} \quad \operatorname{rank} \left(\begin{bmatrix} \mathcal{H}_L(\hat{\mathbf{u}}_{\text{data}}) \\ \mathcal{H}_L(\hat{\mathbf{y}}_{\text{data}}) \end{bmatrix} \right) = L + n \end{aligned}$$

$$\begin{bmatrix} \mathcal{H}_L(\hat{\mathbf{u}}_{\text{data}}) \\ \mathcal{H}_L(\hat{\mathbf{y}}_{\text{data}}) \end{bmatrix} g = \begin{bmatrix} \mathbf{u}_{\text{past}} \\ \mathbf{u} \\ \mathbf{y}_{\text{past}} \\ \mathbf{y} \end{bmatrix}$$

$$\|g\|_0 \leq L + n$$

$$\mathbf{u}_k \in \mathcal{U}_k \quad \forall k$$

$$\mathbf{y}_k \in \mathcal{Y}_k \quad \forall k$$

Step 2: Convex relaxation $\rightarrow$ drop the rank constraint

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{y}, \mathbf{g}, \hat{\mathbf{u}}_{\text{data}}, \hat{\mathbf{y}}_{\text{data}}} \quad & \sum_{k=0}^{K-1} \|\mathbf{y}_k\|_Q^2 + \|\mathbf{u}_k\|_R^2 \\ \text{subject to} \quad & (\hat{\mathbf{u}}_{\text{data}}, \hat{\mathbf{y}}_{\text{data}}) = \operatorname{argmin} \|\hat{\mathbf{u}}_{\text{data}} - \mathbf{u}_{\text{data}}\|^2 + \|\hat{\mathbf{y}}_{\text{data}} - \mathbf{y}_{\text{data}}\|^2 \\ & \begin{bmatrix} \mathcal{H}_L(\hat{\mathbf{u}}_{\text{data}}) \\ \mathcal{H}_L(\hat{\mathbf{y}}_{\text{data}}) \end{bmatrix} \mathbf{g} = \begin{bmatrix} \mathbf{u}_{\text{past}} \\ \mathbf{u} \\ \mathbf{y}_{\text{past}} \\ \mathbf{y} \end{bmatrix} \\ & \|\mathbf{g}\|_0 \leq L + n \\ & \mathbf{u}_k \in \mathcal{U}_k \quad \forall k \\ & \mathbf{y}_k \in \mathcal{Y}_k \quad \forall k \end{aligned}$$

What is the solution to the nested argmin?

Step 3: from sparsity of g to a bound on $\|g\|_1$

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{y}, g} \quad & \sum_{k=0}^{K-1} \|y_k\|_Q^2 + \|u_k\|_R^2 \\ \text{subject to} \quad & \begin{bmatrix} \mathcal{H}_L(\mathbf{u}_{\text{data}}) \\ \mathcal{H}_L(\mathbf{y}_{\text{data}}) \end{bmatrix} g = \begin{bmatrix} \mathbf{u}_{\text{past}} \\ \mathbf{u} \\ \mathbf{y}_{\text{past}} \\ \mathbf{y} \end{bmatrix} \\ & \|g\|_1 \leq \alpha \\ & u_k \in \mathcal{U}_k \quad \forall k \\ & y_k \in \mathcal{Y}_k \quad \forall k \end{aligned}$$

Is it a convex problem now?

Step 4: Replace the bound on $\|g\|_1$ with a regularization term

Regularized data-enabled predictive control

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{y}, \mathbf{g}} \quad & \sum_{k=0}^{K-1} y_k^2 + ru_k^2 + \underbrace{\lambda_g \|\mathbf{g}\|_1}_{\text{regularization parameter}} \\ \text{subject to} \quad & \begin{bmatrix} \mathcal{H}_L(\mathbf{u}_{\text{data}}) \\ \mathcal{H}_L(\mathbf{y}_{\text{data}}) \end{bmatrix} \mathbf{g} = \begin{bmatrix} \mathbf{u}_{\text{past}} \\ \mathbf{u} \\ \mathbf{y}_{\text{past}} \\ \mathbf{y} \end{bmatrix} \\ & u_k \in \mathcal{U}_k \quad \forall k \\ & y_k \in \mathcal{Y}_k \quad \forall k \end{aligned}$$

- ▶ larger λ_g increase robustness to noise but affect the optimality
- ▶ larger λ_g corresponds to selecting a model with smaller n
- ▶ a posteriori, $\|\mathbf{g}\|_0$ informs about the model size

Noise in the real-time measurements

Noise may also be present in the “initial conditions” $\mathbf{u}_{\text{past}}, \mathbf{y}_{\text{past}}$.

- ▶ a noisy sequence of T_{ini} measurements may not be compatible with the data

$$\begin{aligned} \min_{\mathbf{u}, \mathbf{y}, \mathbf{g}, \sigma} \quad & \sum_{k=0}^{K-1} y_k^2 + r u_k^2 + \lambda_g \|\mathbf{g}\|_1 + \lambda_\sigma \|\sigma\|_1 \\ \text{subject to} \quad & \begin{bmatrix} \mathcal{H}_L(\mathbf{u}_{\text{data}}) \\ \mathcal{H}_L(\mathbf{y}_{\text{data}}) \end{bmatrix} \mathbf{g} = \begin{bmatrix} \mathbf{u}_{\text{past}} \\ \mathbf{y}_{\text{past}} + \sigma \end{bmatrix} \\ & u_k \in \mathcal{U}_k \quad \forall k \\ & y_k \in \mathcal{Y}_k \quad \forall k \end{aligned}$$

Handwritten notes and annotations:

- T_{ini} (with a wavy line) points to the $\mathbf{u}_{\text{past}}$ and $\mathbf{y}_{\text{past}} + \sigma$ terms in the constraint matrix.
- “Slack variable σ ” points to the σ term in the constraint matrix.
- Handwritten list on the right: $u_{-T_{\text{ini}}+1}, \dots, u_1, u_0$.
- Handwritten note “ $\rightarrow A, B, C$ ” points to the $\mathcal{H}_L$ terms in the constraint matrix.

We also need to deal with noise in System Identification!

However, there is increasing (and very recent!) evidence that **data-enabled control** deals with noise better than **data $\rightarrow$ sys ID $\rightarrow$ state-space $\rightarrow$ MPC**.

What is hard to do with data-driven control

- ▶ **State constraint**

If a state is known and its value needs to be controlled, then a model-based approach may work better.

- ▶ **Prior/side information**

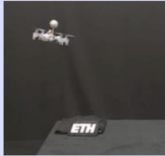
If some system information is available (range of parameters, nominal values, structure of the problem) then sysID can typically incorporate this information.

- ▶ **Nonlinearity**

The theory we have seen today strongly relies on LTI system theory. Extensions to nonlinear systems are complicated.

- ▶ **Computational complexity**

A data representation is less compact than state-space models (what is a model, by the way?). This requires memory and computation time.



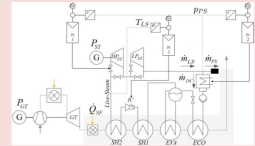
quad coptor fig-8 tracking



quadruped (by Fawcett, Afsari Amers, & Hamed)



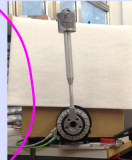
greenhouse automation (by Automatoes)



combined cycle power plant (by P Mahdavi pour. et al)



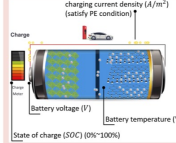
robotic excavator



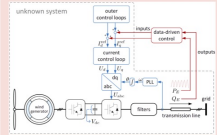
pendulum swing up



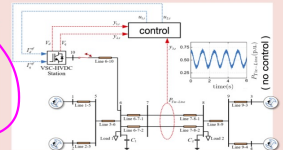
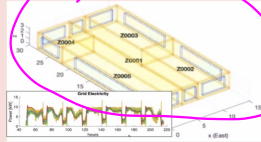
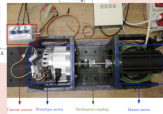
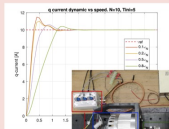
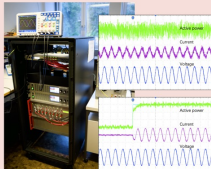
traffic coordination (by J. Wang et al.)



battery charging (by K. Chen et al.)



wind turbine control



Surely, nobody would put apply such a **shaky data-driven method**

- ▶ on the **world's most complex engineered system** (the electric grid),
- ▶ using the **world's biggest actuators** (Gigawatt-sized HVDC links),
- ▶ and subject to **real-time, safety, & stability constraints** ... right?

Dear Linbin and Florian,

I just submitted a very favourable review of your paper [...] which I believe could be of importance to our work at Hitachi Power Grids. We do have [...] require off-line tuning that [...] comissioning engineer can do on his own [...] an adaptive approach would be very interesting.

If possible I would like to try the decentralized DeePC approach with our more detailed HVDC system models on the interarea oscillation problem. Could so some code be made available [...] ? Would you be interested in working together to do such a demonstration ? [...]

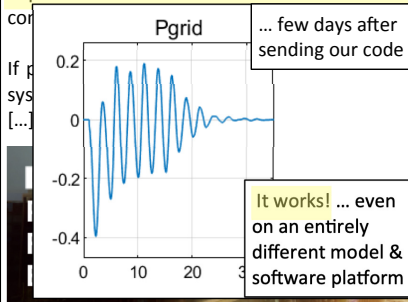


Surely, nobody would put apply such a **shaky data-driven method**

- ▶ on the **world's most complex engineered system** (the electric grid),
- ▶ using the **world's biggest actuators** (Gigawatt-sized HVDC links),
- ▶ and subject to **real-time, safety, & stability constraints** ... right?

Dear Linbin and Florian,

I just submitted a very favourable review of your paper [...] which I believe could be of importance to our work at Hitachi Power Grids. We do have [...] require off-line tuning that [...] an adaptive approach would be very interesting.



and DeePC approach with our more detailed HVDC problem. Could so some code be made available to do such a demonstration ? [...]





This work is licensed under a
[Creative Commons Attribution-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-sa/4.0/)

<https://bsaver.io/COCO>

